CSE 391: System & Software Tools

Welcome!
(and intro to Linux)

Ruth Anderson
✉️ rea@cs.uw.edu
🔗 cs.uw.edu/391

Agenda

1. Logistics & syllabus day stuff
2. What is Unix & Linux (and why do we care)?
3. Live demo: command line interfaces, the shell, and Linux commands

What is 391? (1/2)

Learning objectives: collection of tools and topics not addressed in other classes that we think <u>all</u> CSE majors should know.

- Often called: the “missing semester” or “missing quarter” of a CS major
- Routinely cited (by alums) as one of the most helpful classes for future software engineers, data scientists, etc.
- <u>Not a required class for the major or a prerequisite for any class</u>; the department offers it because **students ask for it!**

Bottom line: this class exists for your benefit!

What is 391? (2/2)

Logistically,

- 1-credit, credit/no-credit class, assessed on weekly homework
  - note: this means this class doesn't impact your GPA (but has other implications)
- Meets once a week (right now!)
- Uses:
  - course website (https://cs.uw.edu/391) for all course materials
  - Ed discussion board
  - Gradescope (for submitting homework)
- Does not use Canvas outside of links to recordings

Course Staff

Instructor:
Ruth Anderson (she/her)

Email: rea@cs.washington.edu

OH: **Mon** 1:30 – 2:30pm (CSE 558 & Zoom)
Tues 11:00 – 12:00pm (CSE 558 & Zoom)

TA:
Helena Kaikkonen (she/her)

Email: helenak@uw.edu

OH: **Mon** 12:30 – 1:30pm, (CSE2 151)
Fri 1:00 – 2:30 pm (CSE2 151)

Class Structure

This class roughly has four components:

1. Assessment (assignments)
2. Pre-recorded lecture content
3. In-class lecture & activities
4. Discussion board & office hours

Assessment: Weekly Homeworks

Your grade is entirely based off of take-home homeworks.

- 10 assignments, each worth (up to) 2 points
- Released after each lecture, due the following Tuesday at 1PM
  - submitted & graded on Gradescope
- Almost entirely autograded, with unlimited submissions & feedback
- **No late assignments accepted**
  - exception: DRS & religious accommodations (please talk to me!)
  - see course syllabus note on extenuating circumstances

Assessment: Course Grade

To receive credit for this class, you need **14** (out of **20**) homework points.

Recommendation: don't skip the early assignments!

- Assignments build off of each other
- Give yourself a buffer
- For transparency: the first few are easier :')

Bonus: one extra credit point if you complete the intro survey before Friday night!

Assessment: Collaboration & Misconduct

Collaboration is encouraged, but all submitted work **must be your own.**

- General hard line: **do not show, use, or discuss solutions**
- Allowed:
  - "Hey, can you show me how this command works again?"
  - "Try using the `tail` command on problem #3"
  - "The problem Ruth went over in class was pretty helpful in doing HW8"
  - "Watch the pre-class video that Matt did on it! It guides you through it!"
- Not allowed:
  - "Here's how I did HW4, take a look (but don't copy)"
  - "Here's my HW4, can you take a look and help me debug?"
- Applies to generative AI in the same way!

For more, see: academic integrity policy & section on generative AI

aside: why this misconduct policy?

Unlike other classes, your grade in this class (mostly) doesn't matter.

But, these are essential topics if your future work involves programming, **and we want you to learn them!**

Learning these things requires some struggle—"skipping" it will hinder your learning & productivity in the long run.

We staff have lots of personal experience and evidence from peers & previous students

But, this is also not meant to be a stressor—if you're struggling, **reach out!**

Pre-recorded Lecture Content

Starting from week 2: ~ 20-45 minutes of pre-recorded lecture content

- Goal: a “first look” at syntax & core content, at your own pace
- Includes “pause & think” questions
- Logistics:
  - will be posted a week in advance on website
  - you should watch each week’s video <u>before</u> attending class

You should treat this as homework, and I will assume that you’ve done them!

Class Sessions

Class will be a mix of:

- More depth on pre-recorded topics (incl. some new content)
- Q&A + review of pre-recorded topics – come with questions!
- “How to do the upcoming homework” – strategies & problem-solving
- Solutions for previous homework (on request)
- If time permits: fun extra content :)

Big picture goal: **<u>actively</u>** engage with course material!

Getting Help: Office Hours & Ed

Synchronous help: come to Ruth or Helena's office hours!

- Can be: content questions, homework help, diving deeper, ...
- Can also be: just do the homework and have someone around

Asynchronous help: post on the Ed discussion board!

- involves your solution code? make a private post.
- otherwise? whatever you're comfortable with!
- feel free to answer each other's questions!

Getting Help: Everything Else

Want to discuss something more privately? Feel free to shoot me an email!

In particular,

- More than happy to work with you to navigate difficult personal situations
- Feel that you're slipping? Let me know **early!**
- Not sure? feel free to ask (politely) – the worst I'll say is no
  - keep in mind: helping you learn this material is literally my job!

Bottom line: **please reach out!**

A Pitch, To You

From our perspective, 391 is the definition of “you get out what you put in”. It’s also an “odd” class in many ways. So, some advice & thoughts:

- We are <u>not</u> expecting this class to be your #1 priority in life!
- but, in order to learn the material, you will need to:
  - **actively engage** (watch the videos, come to class, do the HW)
  - **plan & prioritize** (submit the homeworks – no extensions!)
- This class should not be a major source of stress/work (it’s 1 credit)
  - feeling like it is? **<u>please reach out!</u>**
- but, there’s a ton of room to explore if you’re interested.

okay.

any questions?

Course Topics

- Linux & its command line interface (CLI)
- Shell commands
- Input/output redirection and pipes
- Version control and collaboration using git
- Regular expressions, grep, and sed
- Unix users, groups, and permissions
- Basic data processing
- Basic shell scripting

Operating Systems? (1/2)

An **operating system** is the software that manages software & hardware resources (e.g. computer memory, network connections, peripherals, users).

e.g. Ruth is currently using Windows 11

Most things you think of as “computers” have operating systems, including laptops & desktops, phones, servers, smart watches, …

Operating Systems? (2/2)

What are the most common operating systems for…

- personal computers/devices?
  - **~43% Android + ~17% iOS + ~5% macOS** as of 2024 (StatCounter) – **all Unix-like**
- server computers?
  - **~85% Unix** as of 2023 (W3 Techs)
- supercomputers?
  - **100% Linux** out of the Top 500 fastest supercomputers since 2017 (Top500)

Unix? Linux?

A quick historical sketch:

- **Unix**: proprietary OS developed at Bell Labs in late 1960s - early 1970s
  - some famous people include Dennis Ritchie and Ken Thompson, who also made C!
- **Linux**: group of free & open-source Unix-like OSes, starting in 1991
  - some famous (but arguably not good) people include Linus Torvalds (who also made git) and Richard Stallman
  - related: the GNU project and the "Free Software movement"

In this class: will use them somewhat interchangeably.

(but, the actual difference can be an extremely heated topic)

Why Unix & Linux? (1/2)

Introduced many key ideas:

- **combining many little programs to solve one big problem**
- multiple users (with different permissions)
- hierarchical file system (with recursive folders)
- “everything is a file”
- documentation included with software
- also: shell scripts, coroutines, pipelines, regular expressions, ...

Why Unix & Linux? (2/2)

Widely impactful & influential:

- **easily the most deployed operating system in the world**
- now is realized as:
  - macOS & iOS (Unix -> BSD -> FreeBSD -> Darwin)
  - Android (uses Linux kernel)
  - Linux “distributions”, like Ubuntu, Debian, Arch, Red Hat, and Amazon Linux
  - (and integrated with Windows via WSL)
- among other things, powers most web servers & cloud computing

Bottom line: almost impossible to “dodge” Linux as a programmer!

Shell?

Shell: an interactive program that allows the user to to interact with the operating system & its applications. Almost always text-based.

Why use a shell (instead of a “Graphical User Interface” or GUI)?

- can be much easier & faster to do complicated tasks
- useful for working on remote computers
- highly programmable & customizable
- if you go into software engineering, you’ll have to :’)

Accessing Linux & Unix in 391

Our (roughly ordered) suggestion is:

1. `ssh` (remotely log in) to:
   - `attu` (CSE majors)
     - [New in 26wi, you will need to Download and install Husky OnNet in order to access attu off-campus]
   - `linuxNN` (EE majors)
   - `calgary/cancun` (nomajors taking CSE 351)
   - `vergil/ovid` (all UW students)
2. run Linux on your own computer via the CSE department's virtual machine (free for all UW students in 391)

See: Working at Home page on course website.

Note: we won't officially support other ways of running Linux.

"But it works on my computer" won't apply!

the rest of this is a live demo :)

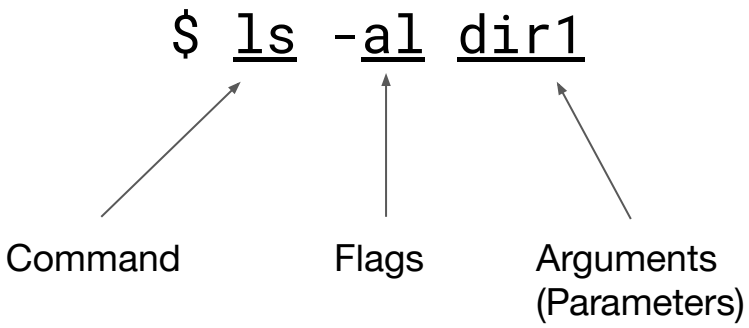
For next week...

1. Fill out EC HW0 Survey by 11:59pm Friday Jan 9
2. Complete HW1 by Tuesday Jan 13 at 1 pm
3. Watch the Week 2 pre-lecture videos **before** next week's lecture

Appendix: Commands

Anatomy of a Command

Command line arguments surprisingly can be inconsistent, but for this class we'll split them up into the **command**, optional **flags**, and optional **arguments.**



Command Line Arguments

Much like methods in Java (or other languages), commands take "**arguments**".

For example, the following line lists all the files inside of the `dir1` folder:

```
$ ls dir1
```

Here, we'll say that `dir1` is an argument to `ls`. Arguments are separated by spaces.

Flags are a special type of argument that typically change the behaviour of a program slightly; they are usually prepended with a `-` or –. For example,

```
$ ls -al dir1
```

Combines two flags: the `-a` flag (list hidden files) and the `-l` flag (use long-list format).

Basic Shell Commands

command	description
`pwd`	**P**rint current **w**orking **d**irectory
`cd`	**C**hange working **d**irectory
`ls`	List files in working directory
`man`	Bring up manual for a command
`exit`	Log out of shell

System Commands

command	description
`clear`	Clears all output from console
`date`	Output the system date
`cal`	Output a text calendar
`uname`	Print information about the current system

Relative Directories

directory	description
`.`	References the working directory
`..`	References the parent of working directory
`~username`	username's **home** directory
`~/Desktop`	Your desktop

Unix File System Conventions

directory	description
`/`	Root directory that contains all directories
`/bin`	Applications/programs (i.e. binaries)
`/dev`	Hardware devices
`/etc`	Configuration files
`/home`	Contains user’s home directories
`/proc`	Running programs (processes)
`/tmp, /var`	Temporary files
`/usr`	Universal system resources

Directory Commands

directory	description
`ls`	List files in working directory
`pwd`	**P**rint current **w**orking **d**irectory
`cd`	**C**hange working **d**irectory
`mkdir`	Make a new directory
`rmdir`	Remove the given directory (must be empty)

File Commands

directory	description
`cp`	Copy a file
`mv`	Move a file (also used to rename files)
`rm`	Remove the given file
`touch`	Create empty file, or change time-modified

Warning: the above commands do **not** ask for confirmation.
Be careful when moving, copying, or deleting files!
(the man pages have flags that prevent this behaviour!)

Text Editors

command	description
`nano`	Very simple editor
`vim`	More advanced text-editor
`emacs`	More advanced text-editor

In many instances, you'll be interacting with a Linux system only through a console – so a command-line text editor is necessary.

vim and emacs are both powerful text editors used by many experienced Linux users; it's up to you to pick one for this class.

vim basics

Key stroke	description
`:w`	Write (save) the current file
`:wq`	Write (save) the current file and exit
`:q!`	Quit, ignoring all changes
`i`	Go into insert mode
`Esc`	Go back to normal mode
`hjkl`	Move cursor left, down, up, right
`u`	Undo last change
`x`	Delete character

emacs basics

C = control key

M = alt/meta key

Key stroke	description
`C-x C-f`	Read a file into emacs
`C-x C-s`	Save a file to disk
`C-x C-c`	Exit emacs
`C-s`	Search forward
`C-r`	Search backwards
`C-v`	Scroll to next screen
`M-v`	Scroll to previous screen
`C-x u`	Undo